

mcr_camera_2026base プログラム設計仕様書

GR-PEACH (RZ/A1H) マイコンカラーリー カメラクラス開発

バージョン 1.1

作成日 2026 年 5 月 3 日

作成者 Shiozawa

ステータス ドラフト

本文書は mcr_camera_2026base プロジェクトのプログラム
設計仕様書ですううううううううう。uuuuuuuuuuuuuu
ベアメタル環境における割り込み駆動アーキテクチャの詳細設計を記述します

改訂履歴

版	日付	著者	変更内容
1.0	2026-02-19	Shiozawa	初版作成
1.1	2026-05-03	Shiozawa	RunController クラス（メイン走行制御）追加。エンコーダ距離制御をタイマー（ms 単位）に置換。

目次

改訂履歴	1
1 概要	3
1.1 用語定義	3
2 背景・課題	3
3 目的	3
4 システムアーキテクチャ	3
4.1 実行フロー図	4
4.2 3 フェーズモデル	4
5 ファイル構成	4
5.1 ディレクトリ構造	5
5.2 ディレクトリ役割	5
5.3 全ファイル一覧	5
5.4 機能とファイルの対応表	6
6 技術スタック	6
7 API 仕様	6
7.1 IModule インターフェース	6
7.2 Encoder クラス	7
7.3 Camera クラス	9
7.4 Trace クラス	10
7.5 Motor クラス	13
7.6 Servo クラス	15
7.7 Onboard クラス	16
7.8 Serial クラス	18
7.9 SDCard クラス	19
7.10 RunController クラス	21
8 実行モデル詳細	25
8.1 割り込みハンドラ	25
8.2 ラッチ方式ドライバの動作原理	26

1 概要

GR-PEACH (RZ/A1H) を使用した、マイコンカーラリー (MCR) カメラクラス開発のためのベースプログラムである。

▷ 設計思想

- mbed OS を使用せず、ルネサス標準の `iodefine.h` を用いたベアメタル（レジスタ直接操作）環境で構築する
- 既存のスパゲッティコードを廃し、クラスベースの `init/update` パターンを採用したモダンなアーキテクチャを目指す

1.1 用語定義

用語	定義
MCR	マイコンカーラリーの略称
GR-PEACH	ルネサス RZ/A1H 搭載の開発ボード
MTU2	マルチファンクション・タイマ・ユニット 2。PWM 出力に使用
ラッチ方式	セッターで値を内部変数に保持し、 <code>update()</code> 呼び出し時にのみハードウェアへ反映する方式
IModule	全モジュール共通の基底インターフェース (§ 7.1)

2 背景・課題

⚠ 開発環境の課題

1. **mbed OS サービス終了**: mbed のサービス終了に伴い、将来的な継続利用や保守が困難になる
2. **開発環境の老朽化**: 既存プロジェクトの e2 studio 環境やコンパイラバージョンが古く、最新の環境でビルドを通す必要がある
3. **ブラックボックス化の解消**: OS 内部処理への依存を減らし、デバッグを容易にする必要がある

3 目的

1. mbed OS に依存しない、RZ/A1H 用の C++ ベースプログラム環境の構築
2. クラス化・モジュール化による可読性・保守性の向上
3. 共通インターフェース (IModule) による統一的な制御フローの確立

4 システムアーキテクチャ

システムはタイマー割り込みによる定期実行（1 ループ/1ms を想定）で完全に同期して動作する。処理は 3 つのフェーズに分離される。

4.1 実行フロー図

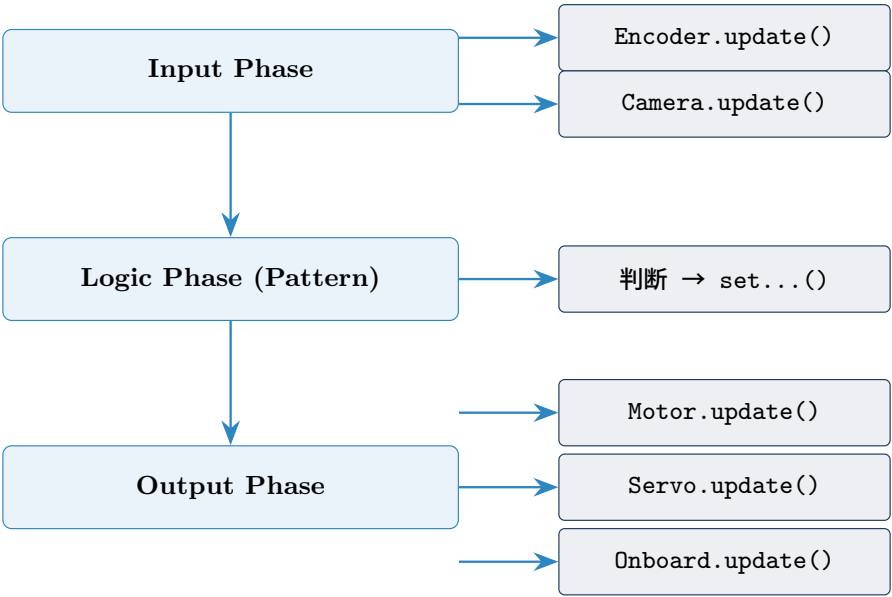


図 1: 割り込みハンドラ実行フロー

4.2 3 フェーズモデル

順序	フェーズ	内容
1	Input Phase	センサ値の読み取り (Encoder, Camera)
2	Logic Phase	判断ロジック実行、各ドライバの <code>set...()</code> を呼び出し (この時点では出力しない)
3	Output Phase	全ドライバの <code>update()</code> を呼び出し、ハードウェアへ反映

▷ ラッチ方式の利点

Logic Phase でセットした値は即座にハードウェアへ反映されない。Output Phase で一括反映することで、割り込み内での出力タイミングを統一し、グリッチや不整合を防止する。

5 ファイル構成

5.1 ディレクトリ構造

```
mcr_camera_test1/  
  src/  
    mcr_camera_test1.cpp ← メイン・割り込みハンドラ  
  core/  
    IModule.h ← 基底インターフェース  
  drivers/  
    Camera.h / Camera.cpp ← カメラ制御  
    Encoder.h / Encoder.cpp ← エンコーダ制御  
    Motor.h / Motor.cpp ← モータ制御  
    Onboard.h / Onboard.cpp ← LED・スイッチ制御  
    SDCard.h / SDCard.cpp ← SD カード制御  
    Serial.h / Serial.cpp ← シリアル通信  
    Servo.h / Servo.cpp ← サーボ制御  
  modules/  
    Trace.h / Trace.cpp ← ライントレースロジック  
  utils/ ← ユーティリティ (予定)  
  generate/  
    iodef.h ← レジスタ定義 (自動生成)  
  doc/  
    main.tex ← 本仕様書
```

5.2 ディレクトリ役割

ディレクトリ	役割
src/core/	システム基盤 (IModule インターフェース)
src/modules/	ロジックモジュール (Trace 等 — 未実装)
src/drivers/	ハードウェアドライバ (Motor, Servo, Encoder, Camera, Serial, SDCard, Onboard)
src/utils/	ユーティリティ関数
generate/	e2 studio 自動生成ファイル (iodef.h 等)
doc/	ドキュメント

5.3 全ファイル一覧

ファイル	役割
src/mcr_camera_2026base.cpp	メインエントリ・割り込みハンドラ・Pattern ロジック
src/core/IModule.h	全モジュール共通の基底インターフェース
src/drivers/Motor.h/.cpp	モータ PWM 制御 (MTU2 ch3/4)
src/drivers/Servo.h/.cpp	サーボ PWM 制御 (MTU2 ch0)
src/drivers/Encoder.h/.cpp	ロータリーエンコーダ読取 (MTU2 位相計数)
src/drivers/Camera.h/.cpp	カメラ画像取得・ライン検出 (CEU/DMA)
src/drivers/Serial.h/.cpp	シリアル通信 (SCIF)
src/drivers/SDCard.h/.cpp	SD カードログ書込み (SPI)

ファイル	役割
src/drivers/Onboard.h/.cpp	オンボード LED・スイッチ制御 (GPIO)
src/modules/Trace.h/.cpp	ライントレース・PID 制御ロジック

5.4 機能とファイルの対応表

機能	メインファイル	関連ファイル
割り込みハンドラ	mcr_camera_2026base.cpp	全ドライバ・モジュール
モータ制御	Motor.cpp	IModule.h, iodefne.h
サーボ制御	Servo.cpp	IModule.h, iodefne.h
エンコーダ	Encoder.cpp	IModule.h, iodefne.h
カメラ	Camera.cpp	IModule.h, iodefne.h
シリアル通信	Serial.cpp	IModule.h, iodefne.h
SD カード	SDCard.cpp	IModule.h, iodefne.h
LED・スイッチ	Onboard.cpp	IModule.h, iodefne.h
ライントレース	Trace.cpp	IModule.h, Camera.h, Encoder.h

6 技術スタック

項目	内容
ターゲットボード	GR-PEACH (ルネサス RZ/A1H, ARM Cortex-A9)
開発環境	Renesas e2 studio
言語	C++ (ベアメタル、mbed OS 不使用)
コンパイラ	GCC ARM (e2 studio 標準)
レジスタ定義	iodefine.h (e2 studio 自動生成)

7 API 仕様

本章では各クラスの公開 API をデータシート形式で記述する。全クラスは IModule インターフェース (§ 7.1) を継承する。

7.1 IModule インターフェース

IModule — 基底インターフェース

ファイル: src/core/IModule.h

種別: 純粋仮想クラス

概要

全てのドライバおよびロジックモジュールが実装すべき共通インターフェース。init() で初期化、update() で周期処理を行う。

機能一覧

- 全モジュール共通の初期化・周期処理メソッドを定義
- 仮想デストラクタによる安全なポリモーフィズム

関数ドキュメント

◆ init()

```
virtual void init() = 0
```

初期化処理。レジスタ設定・変数初期化を行う。システム起動時に 1 回のみ呼び出される。

◆ update()

```
virtual void update() = 0
```

周期処理。割り込みハンドラまたはメインループから定期的に呼び出される。

実装例

```
1 class IModule {
2 public:
3     virtual ~IModule() {}
4     virtual void init() = 0;    // 初期化処理
5     virtual void update() = 0; // 周期処理
6 };
```

Listing 1: IModule インターフェース定義

7.2 Encoder クラス

Encoder — エンコーダドライバ

ファイル: src/drivers/Encoder.h, Encoder.cpp

継承: IModule

Input Phase

概要

MTU2 (チャンネル 1) の位相計数モード 2 を使用し、エンコーダのパルスをカウントする。フィルタリング処理を行い、走行距離と現在の速度を算出・管理する。

機能一覧

- トータル走行距離・コース走行距離の管理
- 現在速度の算出
- 距離の任意設定（クリア機能含む）

内部定数

定数名	値	説明
Filter_N	2	フィルタバッファサイズ
DIST_PER_PULSE	(定数)	1 パルスあたりの移動距離 [mm]

内部変数

変数名	型	初期値	説明
_totalDistance	float	0.0	トータル走行距離
_courseDistance	float	0.0	コース走行距離
_speed	float	0.0	現在速度
_deltaCount	int	0	今回のパルス変動量
_filter_buf []	int []	0	フィルタ用バッファ

関数ドキュメント

◆ init()

```
void init(void)
```

MTU2 の位相計数モード 2 を初期化する。GPIO 設定およびタイマスタートを行う。

◆ update()

```
void update(void)
```

MTU2 カウンタ値を読み取り、前回値との差分から距離・速度を更新する。

◆ getTotalDistance()

```
float getTotalDistance(void)
```

— トータル走行距離 (_totalDistance) を返す。

◆ getCourseDistance()

```
float getCourseDistance(void)
```

— コース走行距離 (_courseDistance) を返す。

◆ getSpeed()

```
float getSpeed(void)
```

— 現在速度 (_speed) を返す。

◆ setTotalDistance(dist)

void setTotalDistance(float dist) — トータル走行距離を任意の値に設定する (0 でクリア)。

◆ setCourseDistance(dist)

void setCourseDistance(float dist) — コース走行距離を任意の値に設定する (0 でクリア)。

7.3 Camera クラス

Camera — カメラ制御および画像処理ドライバ

ファイル: src/drivers/Camera.h, Camera.cpp

継承: IModule

Input Phase

概要

関数ドキュメント (画像処理)

◆ resize(src, dst, percent)

void resize(uint8_t *src, uint8_t *dst, double percent)

(旧: ImageReduction) — 画像を縮小する。

◆ extractBrightness(src, dst)

void extractBrightness(uint8_t *src, uint8_t *dst)

(旧: Extraction_Brightness) — YCbCr 画像から輝度成文 (Y) を抽出する。

◆ binarize(src, dst, threshold)

void binarize(uint8_t *src, uint8_t *dst, int threshold)

(旧: Binarization) — 画像を指定閾値で二値化する。

◆ percentileMethod(src, percent)

int percentileMethod(uint8_t *src, int percent)

(旧: Percentile_Method) — パーセンタイル法による閾値決定を行う。

◆ discriminantAnalysis(src)

int discriminantAnalysis(uint8_t *src)

(旧: DiscriminantAnalysis_Method) — 判別分析法による閾値決定を行う。

◆ extractPart(src, x, y, dst, w, h)

```
void extractPart(uint8_t *src, int x, int y, uint8_t *dst, int w, int h)
```

(旧: Image_part_Extraction) — 画像の一部を切り出す。

◆ standardDeviation(...)

```
double standardDeviation(uint8_t *data, double *devi, int w, int h)
```

(旧: Standard_Deviation) — 標準偏差を算出する。

◆ covariance(...)

```
double covariance(double *deviA, double *deviB, int w, int h)
```

(旧: Covariance) — 共分散を算出する。

◆ matchPattern(...)

```
void matchPattern(uint8_t *src, ImagePartPattern *tmplt, ...)
```

(旧: PatternMatching_process) — パターンマッチングを行う。

⚠ 実装上の注意

- DisplayBase ライブラリのセットアップ手順は複雑なため、既存実装 (init_Camera) を参考に移植する。
- 画像処理関数は重いため、処理時間オーバーに注意が必要。

7.4 Trace クラス

Trace — ライントレースモジュール

ファイル: src/modules/Trace.h, Trace.cpp

継承: IModule

Logic Phase

概要

カメラ画像からライン位置を検出し、PID 制御によりステアリング角度を算出する。update() 内で検出→PID 計算の一連処理を実行する。

機能一覧

- 画像二値化によるライン検出
- PID 制御によるステアリング角度算出
- ゲインパラメータの動的設定
- ステアリング角度のリミット処理

内部定数

定数名	値	説明
THRESHOLD	未定	二値化閾値
STEER_MAX	未定	ステアリング角度上限
STEER_MIN	未定	ステアリング角度下限

内部変数

変数名	型	初期値	説明
_kp	float	0.0f	比例ゲイン
_ki	float	0.0f	積分ゲイン
_kd	float	0.0f	微分ゲイン
_integral	float	0.0f	積分値
_prevErr	float	0.0f	前回偏差
_steer	float	0.0f	算出ステアリング角度
_linePos	float	0.0f	検出ライン位置
_lineFound	bool	false	ライン検出成否

関数ドキュメント

◆ <code>init()</code> <code>void init(void)</code> PID パラメータおよび内部状態変数を初期化する。
◆ <code>update()</code> <code>void update(void)</code> ライン検出→PID 計算の一連処理を実行する。内部で <code>binarize()</code> → <code>detectLine()</code> → <code>calcPid()</code> → <code>clampSteer()</code> の順に呼び出す。
◆ <code>getSteer()</code> <code>float getSteer(void)</code> — 算出されたステアリング角度を返す。
◆ <code>getLinePos()</code> <code>float getLinePos(void)</code> — 検出されたライン位置を返す。
◆ <code>isLineFound()</code> <code>bool isLineFound(void)</code> — ラインが検出されたかどうかを返す。

◆ setKp(v)

void setKp(float v) — 比例ゲインを設定する。 $(v \geq 0)$

◆ setKi(v)

void setKi(float v) — 積分ゲインを設定する。 $(v \geq 0)$

◆ setKd(v)

void setKd(float v) — 微分ゲインを設定する。 $(v \geq 0)$

◆ detectLine(data) [内部]

float detectLine(uint8_t* data)

二値化済みの画像データからライン中心位置を算出する。

引数: data (uint8_t*) — 二値化済み画像データ

戻り値: float — ライン中心位置

◆ calcPid(error) [内部]

float calcPid(float error)

PID 制御量を計算する。積分値 (_integral) と前回偏差 (_prevErr) を内部で更新する。

引数: error (float) — 目標位置との偏差

戻り値: float — PID 制御量

◆ binarize(data) [内部]

void binarize(uint8_t* data)

画像データを閾値 (THRESHOLD) に基づいて二値化する。入力バッファを直接書き換える。

引数: data (uint8_t*) — 画像データ (in-place 変換)

◆ clampSteer(val) [内部]

float clampSteer(float val)

ステアリング角度を STEER.MIN~STEER.MAX の範囲にクランプする。

引数: val (float) — クランプ前のステアリング角度

戻り値: float — クランプ後のステアリング角度

7.5 Motor クラス

Motor — モータ PWM ドライバ

ファイル: src/drivers/Motor.h, Motor.cpp

継承: IModule

Output Phase

概要

MTU2 チャンネル 3/4 を使用したリセット同期 PWM モードで左右モータを制御する。ラッチ方式を採用し、`setPower()` でデューティ比を内部保持、`update()` でレジスタへ反映する。

機能一覧

- MTU2 リセット同期 PWM モードによるモータ駆動
- 左右独立のデューティ比制御 (−1.0 ~ 1.0)
- 符号による正転/逆転の方向制御
- ラッチ方式による安全な出力タイミング

内部定数

定数	値	説明
PWM_CYCLE	33332	PWM 周期カウンタ値 (1ms @ P0φ/1)

内部変数

変数名	型	初期値	説明
_pwmLeft	float	0.0f	左モータデューティ比 (−1.0 ~ 1.0)
_pwmRight	float	0.0f	右モータデューティ比 (−1.0 ~ 1.0)

関数ドキュメント

◆ `init()`

`void init(void)`

GPIO 方向ピンおよび MTU2 チャンネル 3/4 のレジスタを初期化する。リセット同期 PWM モードを設定し、カウンタ動作を開始する。

◆ `update()`

`void update(void)`

内部変数のデューティ比をハードウェアへ反映する。内部で `setDirection()` → `calcDuty()` → `applyPwm()` の順に呼び出す。

◆ setPower(l, r)

```
void setPower(float l, float r)
```

左右モータのデューティ比を内部変数に保存する（ラッチ方式）。この時点ではハードウェアへの反映は行わない。

- l (float) — 左モータデューティ比 (−1.0 ~ 1.0、負値で逆転)
- r (float) — 右モータデューティ比 (−1.0 ~ 1.0、負値で逆転)

◆ calcDuty(pwm) [内部]

```
uint16_t calcDuty(float pwm)
```

デューティ比（浮動小数点数）を PWM カウント値に変換する。 $duty = \lfloor |pwm| \times PWM_CYCLE \rfloor$

引数: pwm (float) — デューティ比 (−1.0 ~ 1.0)

戻り値: uint16_t — PWM カウント値

◆ setDirection(l, r) [内部]

```
void setDirection(float l, float r)
```

デューティ比の符号に基づき、方向ピン (GPIO P4.6, P4.7) を設定する。正值で正転 (Low)、負値で逆転 (High)。

- l (float) — 左モータデューティ比
- r (float) — 右モータデューティ比

◆ applyPwm(l, r) [内部]

```
void applyPwm(uint16_t l, uint16_t r)
```

MTU2 レジスタ (TGRA_4/TGRC_4, TGRB_4/TGRD_4) へ PWM 値を書き込む。

- l (uint16_t) — 左モータ PWM カウント値
- r (uint16_t) — 右モータ PWM カウント値

ハードウェアマッピング

信号	ピン	レジスタ	説明
左モータ PWM	P4.4 (TIOC4A)	TGRA_4 / TGRC_4	デューティ比
右モータ PWM	P4.5 (TIOC4B)	TGRB_4 / TGRD_4	デューティ比
左モータ 方向	P4.6 (GPIO)	GPIO.P4 bit6	Low: 正転 / High: 逆転
右モータ 方向	P4.7 (GPIO)	GPIO.P4 bit7	Low: 正転 / High: 逆転

7.6 Servo クラス

Servo — サーボ PWM ドライバ

ファイル: src/drivers/Servo.h, Servo.cpp

継承: IModule

Output Phase

概要

MTU2 チャンネル 0 を PWM モード 1 で駆動し、ステアリングサーボの角度を制御する。ラッチ方式を採用し、`setAngle()` で角度を内部保持、`update()` でレジスタへ反映する。

機能一覧

- MTU2 PWM モード 1 によるサーボ駆動
- 角度→パルス幅の自動変換
- ラッチ方式による安全な出力タイミング

内部定数

定数名	値	説明
SERVO_CENTER	3090	サーボ中心位置のパルス幅カウント値
HANDLE_STEP	23	1 度あたりのカウント値

内部変数

変数名	型	初期値	説明
_angle	float	0.0f	サーボ角度 (度)

関数ドキュメント

◆ `init()`

`void init(void)`

GPIO および MTU2 チャンネル 0 のレジスタを初期化する。PWM モード 1 を設定し、カウント動作を開始する。

◆ `update()`

`void update(void)`

内部変数の角度値をハードウェアへ反映する。内部で `calcPulse()` → `applyServo()` の順に呼び出す。

◆ setAngle(angle)

```
void setAngle(float angle)
```

サーボ角度を内部変数に保存する（ラッチ方式）。0 が中心、正負で左右に振れる。

引数: angle (float) — サーボ角度 (度、機体依存)

◆ calcPulse(angle) [内部]

```
uint16_t calcPulse(float angle)
```

角度をパルス幅カウント値に変換する。 $val = \lfloor angle \times 23 \rfloor + \text{SERVO_CENTER}$

引数: angle (float) — サーボ角度 (度)

戻り値: uint16_t — パルス幅カウント値

◆ applyServo(val) [内部]

```
void applyServo(uint16_t val)
```

MTU2 レジスタ (TGRB_0 / TGRD_0) へパルス幅を書き込む。

引数: val (uint16_t) — パルス幅カウント値

ハードウェアマッピング

信号	ピン	レジスタ	説明
サーボ PWM	P4_0 (TIOC0A)	TGRB_0 / TGRD_0	パルス幅 (立ち下がリエッジ)

7.7 Onboard クラス

Onboard — オンボード LED / スイッチ ドライバ

ファイル: src/drivers/Onboard.h, Onboard.cpp

継承: IModule

Output Phase

概要

GR-PEACH ボード上の RGB LED (3 個) + ユーザー LED (1 個) およびユーザースイッチ (1 個) を制御する。LED はラッチ方式、スイッチは即時読み取り。

機能一覧

- RGB LED 3 個 + ユーザー LED 1 個の制御
- ラッチ方式による LED 出力
- ユーザースイッチの即時読み取り

内部変数

変数名	型	初期値	説明
_ledState[4]	int[4]	{0,0,0,0}	各 LED 状態 (0: 消灯, 1: 点灯)

関数ドキュメント

◆ init()

```
void init(void)
```

GPIO 初期化を行う。全 LED を消灯し、スイッチを入力モードに設定する。PIBC6 の入力バッファ有効化を含む。

◆ update()

```
void update(void)
```

内部変数の LED 状態を GPIO へ反映する。内部で writeLed() を各 LED ID に対して呼び出す。

◆ setLed(id, val)

```
void setLed(int id, int val)
```

指定 LED の状態を内部変数に保存する（ラッチ方式）。

- id (int) — LED 番号 (0:Red, 1:Green, 2:Blue, 3:User)
- val (int) — 0: 消灯, 1: 点灯

◆ sw()

```
int sw(void)
```

ユーザースイッチの状態を即時読み取りで返す。ラッチ方式ではなく、呼び出し時に GPIO を直接読む。内部で readSw() を呼び出す。

戻り値: int — 1: 押下, 0: 解放

◆ writeLed(id, val) [内部]

```
void writeLed(int id, int val)
```

指定 LED の GPIO を直接操作する。RGB LED は Low Active、ユーザー LED は High Active。

- id (int) — LED 番号
- val (int) — 論理的な点灯/消灯状態

◆ readSw() [内部]

```
int readSw(void)
```

SW 用 GPIO (P6.0) を読み取り、論理値に変換して返す。Low = 押下。

戻り値: int — 1: 押下, 0: 解放

ハードウェアマッピング

デバイス	ID	ピン	論理	説明
LED Red	0	P6_13	Low Active	赤色 LED
LED Green	1	P6_14	Low Active	緑色 LED
LED Blue	2	P6_15	Low Active	青色 LED
LED User	3	P6_12	High Active	ユーザー LED
SW User	–	P6_0	Low = 押下	プルアップ、GND 接続

⚠ GPIO 注意事項

- RGB LED は **Low Active**（GPIO Low で点灯）
- スイッチ読み取りには PIBC6 の入力バッファ有効化が必須
- `sw()` はラッチ方式ではなく、呼び出し時に直接 GPIO を読む

7.8 Serial クラス

Serial — シリアル通信ドライバ

ファイル: src/drivers/Serial.h, Serial.cpp

継承: IModule

Logic/Output Phase

概要

SCIF (Serial Communication Interface) を使用してデバッグ用シリアル通信を行う。他のドライバとは異なり、ラッチ方式 (`update()` による遅延送出) は採用せず、`printf` 呼び出し時に**即時出力 (ポーリング送信)**を行う。これにより、途中クラッシュ時にもデバッグ出力を確実に得られるようにする。

機能一覧

- SCIF2 によるシリアル通信 (GR-PEACH USB シリアル通信ポート想定)
- 書式付き出力 (`printf`)
- **即時出力** (バッファリングなし、ポーリング待機)

内部定数

定数名	値	説明
BAUD_RATE	115200	ボーレート

内部変数

特になし (バッファ状態を持たない)。

関数ドキュメント

◆ init()

```
void init(void)
```

SCIF の初期化（ピン設定、モジュールストップ解除、ボーレート設定等）を行う。

◆ update()

```
void update(void)
```

IModule インターフェースを満たすためのダミー関数。何もしない。

◆ printf(fmt, ...)

```
void printf(const char* fmt, ...)
```

書式付き文字列を即時送信する。vsnprintf を内部で使用し、putChar() を経由して文字を送信する。

引数: fmt (const char*) — 書式文字列 (printf 互換)

◆ putChar(c) [内部]

```
void putChar(char c)
```

1 文字を即時書き込む。FIFO/送信レジスタに空きができるまで待機（ポーリング）してから送信する。

7.9 SDCard クラス

SDCard — SD カードログドライバ

ファイル: src/drivers/SDCard.h, SDCard.cpp

継承: IModule

Output Phase

概要

SPI 経由で SD カードへログデータを書き込む。データはバッファリングし、update() でブロック単位で書き出す。

機能一覧

- SPI 経由の SD カードアクセス
- バッファリングによる効率的なブロック書込み
- ログ文字列の追記
- 明示的フラッシュ・クローズ操作

内部定数

定数名	値	説明
BLOCK_SIZE	512	SD ブロックサイズ (bytes)
WRITE_BUF_SIZE	1024	書込みバッファサイズ (bytes)

内部変数

変数名	型	初期値	説明
_writeBuf[]	uint8_t[]	0	書込みバッファ
_bufPos	uint16_t	0	バッファ書込み位置
_isReady	bool	false	SD 初期化完了フラグ
_blockAddr	uint32_t	0	現在の書込みブロックアドレス

関数ドキュメント

◆ <code>init()</code> <code>void init(void)</code> SPI 初期化および SD カードの初期化シーケンス (CMD0 → CMD8 → ACMD41) を実行する。初期化成功で <code>_isReady</code> を <code>true</code> に設定。
◆ <code>update()</code> <code>void update(void)</code> バッファが 1 ブロック分 (512B) 以上溜まっている場合、SD カードへブロック単位で書き出す。内部で <code>writeBlock()</code> を呼び出す。
◆ <code>log(str)</code> <code>void log(const char* str)</code> ログ文字列をバッファに追加する。 引数: <code>str</code> (<code>const char*</code>) — ログ文字列
◆ <code>flush()</code> <code>void flush(void)</code> バッファ内容を即時書き出す。残りデータが 512B 未満でもパディングして書き出す。
◆ <code>close()</code> <code>void close(void)</code> <code>flush()</code> を呼び出した後、ファイル終端処理を行う。SD 通信を終了し、<code>_isReady</code> を <code>false</code> に設定する。

◆ spiWrite(byte) [内部]

```
uint8_t spiWrite(uint8_t byte)
```

SPI 1 バイト送受信を行う。送信と同時に受信データを返す。

引数: byte (uint8_t) — 送信バイト

戻り値: uint8_t — 受信バイト

◆ sendCmd(cmd, arg) [内部]

```
uint8_t sendCmd(uint8_t cmd, uint32_t arg)
```

SD カードコマンドを送信し、レスポンスを返す。

- cmd (uint8_t) — SD コマンド番号
- arg (uint32_t) — コマンド引数

戻り値: uint8_t — R1 レスポンス

◆ writeBlock(data) [内部]

```
bool writeBlock(const uint8_t* data)
```

512B ブロックをアドレス _blockAddr に書き込む。成功で _blockAddr をインクリメント。

引数: data (const uint8_t*) — 書き込みデータ (512B)

戻り値: bool — 書き込み成功/失敗

◆ waitReady() [内部]

```
bool waitReady(void)
```

SD カードのビジー状態が解除されるまで待機する。タイムアウト時は false を返す。

戻り値: bool — ビジー解除成功/タイムアウト

7.10 RunController クラス

RunController — メイン走行制御モジュール（初心者向けベース）

ファイル: src/drivers/RunController.h, RunController.cpp

継承: IModule

Logic Phase

概要

参考プロジェクト mcr_shiozawa_cclass_2.38m-s/main.cpp の intTimer() 内 switch(pattern) 状態遷移を移植したもの。オリジナルでは「エンコーダ累積パルス」で距離管理していたが、本実装では初心者を読みやすいようにすべて「経過時間 (ms)」で管理する。各状態に入るたびに stateMs_ が 0 クリアされ、1ms 割り込みごとに 1 ずつ加算される。

機能一覧

- Pattern 番号による状態遷移（参考プロジェクトと同じ番号体系）
- スタートバー検出 → 通過 → 通常トレース → クランク/レーンチェンジ
- 各状態の遷移条件は経過時間（ms）で判定（距離不要）
- LineDetector の検出フラグを参照してクランク/ハーフラインを判別
- Motor / Servo / Onboard を直接制御
- 走行終了時に isFinished() で外部に通知（main 側でログ保存をトリガ）

状態遷移 (Pattern)

番号	Pattern 名	役割
0	WAIT_SW	スイッチ入力待ち
1	APPROACH_BAR	スタートバーへ前進
2	WAIT_AT_BAR	バー手前で停止
3	ACCEL_AFTER_BAR	バー上昇後の加速
4	CHECK_BAR_OPEN	バー開放確認
10	WAIT_BAR_OPEN	バー開放待ち
11	TRACE_NORMAL	通常トレース
21-23	CROSSLINE_*	クロスライン検出→読み飛ばし→クランク選択
30-31	LEFT_CRANK_*	左クランク
40-41	RIGHT_CRANK_*	右クランク
51-57	RIGHT_HALF_* / RIGHT_LANE_*	右ハーフライン→レーンチェンジ
61-67	LEFT_HALF_* / LEFT_LANE_*	左ハーフライン→レーンチェンジ
101	FINISH	走行終了（ログ保存待ち）

タイマー定数

定数名	値 [ms]	説明
T_LINE_SKIP_MS	100	クロス/ハーフ通過時間
T_CRANK_MS	440	クランク旋回時間
T_LANE_TURN_MS	340	レーンチェンジ寄せ時間
T_LANE_STRAIGHT_MS	0	レーンチェンジ直進時間
T_LANE_COUNTER_MS	250	レーンチェンジ戻し時間
T_COURSE_TIMEOUT_MS	60000	コース 1 周想定タイムアウト
T_BAR_WAIT_PRE_MS	500	バー前進開始までの待機
T_BAR_WAIT_POST_MS	1000	バー手前停止後の待機
T_AFTER_BAR_GO_MS	100	バー後加速の継続時間

走行パラメータ定数

定数名	値	説明
CRANK_HANDLE	42	クランク旋回角度 [度]
CRANK_MOTOR_OUT/IN	100	クランク内外輪モーター [%]
LANE_HANDLE	23	レーンチェンジ寄せ角度 [度]
LANE_COUNTER_HANDLE	38	レーンチェンジ戻し角度 [度]
LANE_MOTOR_IN/OUT	37/55	レーン寄せ時の内/外輪 [%]
LANE_STRAIGHT_MOTOR	100	レーン直進時のモーター [%]
LANE_COUNTER_MOTOR_IN/OUT	20	レーン戻し時の内/外輪 [%]
BRAKE_TARGET_MOTOR	43	ブレーキ走行時の目標出力 [%]
TRACE_ROW	45	通常トレース時の偏差取得行
APPROACH_ROW	95	スタート時の偏差取得行

内部変数

変数名	型	初期値	説明
pattern_	Pattern	WAIT_SW	現在の走行パターン
stateMs_	unsigned long	0	現パターンに入ってから経過時間 [ms]
totalMs_	unsigned long	0	スタート (pattern==11) からの累積時間 [ms]
cnt1Ms_	unsigned long	0	スタートバー処理用の補助タイマー [ms]
finished_	bool	false	走行終了フラグ
handleVal_	int	0	現在のハンドル指示値 [度]
traceOffset_	int	0	ハンドル計算オフセット
leftMotor_, rightMotor_	int	0	通常時の左右モーター指示値
leftBrake_, rightBrake_	int	0	ブレーキ時の左右モーター指示値

関数ドキュメント

◆ init()

```
void init(void) override
```

内部状態を全て初期化し、WAIT_SW 状態へ遷移する。モーター停止・ハンドル中央・LED 消灯を実行。

◆ update()

```
void update(void) override
```

1ms 割り込み (OSTM0) から呼び出される周期処理。stateMs_ と cnt1Ms_ をインクリメントし、現在の pattern_ に応じて内部の run*() メソッドを呼び出す。状態遷移時には stateMs_ が 0 クリアされる。

◆ isFinished()

```
bool isFinished(void) const
```

走行終了状態 (FINISH) に到達したかを返す。main 側がこの戻り値で g_sdlogger.saveToSD() のトリガとして使用する。

戻り値: bool — true=終了済み、false=走行中

◆ getPattern()

```
int getPattern(void) const
```

現在の走行パターン番号を整数で返す (デバッグ表示・ログ用)。

◆ getHandleVal()

```
int getHandleVal(void) const
```

直前に calcHandle() で計算されたハンドル指示値を返す (ログ用)。

◆ forceStop()

```
void forceStop(void)
```

緊急停止用: モーター停止・ハンドル中央・赤 LED 点灯を行い、FINISH 状態へ遷移する。

距離 → タイマー置換の対応表

参考プロジェクト (エンコーダ距離) → 本実装 (経過時間 ms) の対応:

参考プロジェクト	本実装	備考
encoder.clear()	stateMs_ = 0	状態遷移時に自動クリア
encoder.getTotalCount() >= N	stateMs_ >= N	1pulse ≈ 1ms 換算
encoder.getCourseCount() >= ONEM*58	totalMs_ >= 60000	60 秒で終了
encoder.getCnt() <= lowSpeedLimit	(削除)	速度判定はカット

使用例

```
1 // main 起動時
2 g_runController.init();
3
4 // OSTMO 1ms 割り込みコールバック内
5 extern "C" void ostm0_interrupt_callback(void) {
6     g_camera.update();
7     g_motor.update();
8     g_servo.update();
9     g_runController.update(); // 状態遷移を1毎に進めるms
10    g_onboard.update();       // GPIO ラッチ反映
11 }
12
13 // メインループ
14 while (1) {
15     if (g_camera.isFrameReady()) {
16         g_camera.clearFrameReady();
17         g_lineDetector.update(); // ライン検出はフレーム完了時のみ実行
18     }
19     if (g_runController.isFinished()) {
20         g_sdlogger.saveToSD(); // 走行終了時にログ書き出し
21         break;
22     }
23 }
```

Listing 2: RunController の組み込み例

8 実行モデル詳細

8.1 割り込みハンドラ

割り込みハンドラ関数内で各モジュールの `update()` を呼び出す。Pattern 処理はクラス化せず、ハンドラ内にベタ書き（または関数化）する。

```
1 // グローバルインスタンス
2 static Onboard onboard;
3 static Motor motor;
4 static Servo servo;
5
6 void interrupt_handler() {
7     // --- Input Phase ---
8     // encoder.read();
9     // camera.read();
10
11    // --- Logic Phase (Pattern) ---
12    int sw = onboard.sw();
13    if (sw) {
14        motor.setPower(0.2f, 0.2f); // PWM 20%
15        servo.setAngle(10.0f);      // 度10
16        onboard.setLed(0, 1);       // Red ON
```

```
17     onboard.setLed(1, 1);          // Green ON
18     onboard.setLed(2, 1);          // Blue ON
19 } else {
20     motor.setPower(0.0f, 0.0f);
21     servo.setAngle(0.0f);
22     onboard.setLed(0, 1);          // Red ON
23     onboard.setLed(1, 0);          // Green OFF
24     onboard.setLed(2, 0);          // Blue OFF
25 }
26
27 // --- Output Phase ---
28 motor.update();
29 servo.update();
30 onboard.update();
31 // serial.update();
32 // sd.update();
33 }
```

Listing 3: 割り込みハンドラ実装例

8.2 ラッチ方式ドライバの動作原理



図 2: ラッチ方式ドライバのデータフロー